Operating Systems: Memory Management & Paging — Address Binding and Memory Spaces

1.1 Logical versus Physical Address Space

An address generated by the CPU is commonly referred to as a logical address or virtual address, whereas an address seen by the memory unit—that is, the address loaded into the memory address register (MAR) of the memory—is commonly referred to as a physical address.

The compile-time and load-time address-binding methods generate identical logical and physical addresses. However, execution-time address-binding schemes result in differing logical and physical addresses. The run-time mapping from virtual to physical addresses is handled by a hardware device called the Memory Management Unit (MMU).

1.2 Dynamic Relocation & Relocation Register

The simplest MMU scheme uses a relocation register (or base register). The value in the relocation register is added to every logical address generated by a user process at the time the address is sent to memory. The user program never sees real physical memory addresses.

Dynamic loading ensures that a routine is not loaded until it is called, which improves memory-space utilization. Dynamic linking postpones the linking of libraries until execution time, allowing libraries to be shared among multiple processes via shared stubs.

Operating Systems: Memory Management & Paging — Contiguous Allocation & Fragmentation

2.1 Contiguous Allocation Strategies

In contiguous memory allocation, each process is contained in a single contiguous section of memory. Memory is usually partitioned into fixed or variable-sized partitions. In variable-partition schemes, the operating system keeps a table indicating which parts of memory are available and which are occupied.

When allocating a block from available free holes, three primary dynamic storage-allocation algorithms are used: First-Fit allocates the first hole that is big enough; Best-Fit allocates the smallest hole that is big enough; Worst-Fit allocates the largest available hole. First-Fit and Best-Fit are generally faster and produce better memory utilization than Worst-Fit.

2.2 Internal and External Fragmentation

External fragmentation exists when there is enough total memory space to satisfy a request, but the available spaces are not contiguous: storage is fragmented into a large number of small holes. The 50-percent rule states that for First-Fit, given N allocated blocks, 0.5N blocks will be lost to fragmentation.

Internal fragmentation occurs when memory is allocated in fixed-size blocks and the process requires less memory than the block size; the unused portion within the allocated block is wasted. Compaction is a solution to external fragmentation where memory contents are rearranged to place all free memory together in one large block, but it is possible only if relocation is dynamic and done at execution time.

Operating Systems: Memory Management & Paging — Paging Architecture & Translation

3.1 Paging Mechanism

Paging is a memory-management scheme that permits the physical address space of a process to be noncontiguous. Paging avoids external fragmentation and the need for compaction. Physical memory is broken into fixed-sized blocks called frames, and logical memory is broken into blocks of the same size called pages.

Every address generated by the CPU is divided into two parts: a page number (p) and a page offset (d). The page number is used as an index into a per-process page table. The page table contains the base address of each page in physical memory (frame number f), which is combined with the page offset to define the physical memory address.

3.2 Hardware Support and Page Table Base Register

The page table is kept in main memory, and a Page-Table Base Register (PTBR) points to the starting location of the page table in memory. Changing page tables requires changing only this one register, substantially reducing context-switch time.

However, the PTBR scheme has a serious disadvantage: accessing a memory location requires two memory accesses—one to fetch the page-table entry and one to fetch the actual desired data. This doubles the memory access latency unless specialized hardware caching is implemented.

Operating Systems: Memory Management & Paging — Translation Lookaside Buffer (TLB) & Multi-Level Paging

4.1 Translation Lookaside Buffer (TLB)

To overcome the two-memory-access penalty of page tables, computer hardware uses a special, small, fast-lookup hardware cache called a Translation Lookaside Buffer (TLB). The TLB is associative, high-speed memory composed of keys and values. When a virtual address is presented to the MMU, the page number is compared simultaneously against all TLB entries.

If the page number is found in the TLB, a TLB hit occurs, and the frame number is immediately available. If the page number is not found, a TLB miss occurs; the MMU accesses the page table in main memory, retrieves the frame number, and updates the TLB. Effective Memory Access Time (EMAT) is computed as: EMAT = (Hit_Ratio * (TLB_Search_Time + Memory_Access_Time)) + ((1 - Hit_Ratio) * (TLB_Search_Time + 2 * Memory_Access_Time)).

4.2 Hierarchical and Inverted Page Tables

Most modern computers support a large logical address space (e.g., 2^32 or 2^64 bytes). In such systems, the page table itself becomes excessively large. One solution is two-level paging (hierarchical paging), where the page table itself is paged.

An Inverted Page Table has one entry for each real page of physical memory (frame), rather than one entry for each page of logical address space. Each entry contains the virtual address of the page stored in that frame, along with information about the process that owns the page, substantially reducing memory consumption.

Operating Systems: Memory Management & Paging — Virtual Memory and Page Replacement

5.1 Virtual Memory and Demand Paging

Virtual memory involves the separation of logical memory as perceived by users from physical memory. This separation allows an extremely large virtual memory to be provided for programmers when only a smaller physical memory is available.

In Demand Paging, pages are loaded only when they are demanded during program execution. If a process attempts to access a page marked invalid in the page table, a page fault trap occurs. The operating system handles the trap by finding a free frame in physical memory, reading the desired page from secondary backing storage into the frame, updating the page table to valid, and restarting the instruction.

5.2 Page Replacement Algorithms & Thrashing

When no free frame is available, page replacement is required. The FIFO algorithm replaces the oldest page; it is simple but subject to Belady's Anomaly, where page fault rates increase as allocated frames increase. The Optimal (OPT) algorithm replaces the page that will not be used for the longest period of time; it is provably optimal but impossible to implement in practice because it requires future knowledge.

The Least Recently Used (LRU) algorithm associates with each page the time of that page's last use; when a page must be replaced, LRU chooses the page that has not been used for the longest period. Thrashing occurs when a process spends more time paging (swapping pages in and out) than executing instructions, causing CPU utilization to collapse.